

Why tools exist: context beyond the prompt

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 10 of 12 · English notes

Session 10 (24 August 2026) opened with a recap of **context management**. Vijay's framing: every GenAI or agentic app has to prepare a context channel, send it to the LLM, and consume the response. Prompt engineering is one way to shape that channel. **Tool use** is another: the model can request live or proprietary data that was never in the training set.

What this chapter is for

Before the Messages API schema, you need the product reason tools exist. Claude does not magically know Bangalore's weather, a live share price, or an employee record in a UST system. Those facts live outside the model. A tool is a function your application owns; Claude decides *when* to call it and with which arguments; your code actually runs it.

CLASSROOM UI VERSUS DOCS

Notebooks, SDK class names, and model IDs in the August 2026 recording can drift. Treat **tool use** on docs.claude.com as the exam source of truth.

Five ways to enhance context

The trainer listed a stack of techniques that all serve the same job: make the request the model sees richer and more accurate than a raw user sentence.

Technique	What it adds to context	Typical exam angle
Prompt engineering	Structured instructions, examples, roles	How you write the message, not how you fetch data
Tool use	Real-time or proprietary function results	Model requests a named tool; the app executes it
RAG	Retrieved chunks from a corpus	Do not dump a 10,000-page PDF into one prompt
MCP	Standardised access to third-party tools and data	Write the integration once; reuse across clients
Memory	Prior turns or longer-lived state	Conversational apps need an explicit memory design

These are complementary, not rivals. A hiring agent might use prompts for tone, tools for HR APIs, RAG for policy PDFs, MCP for a vendor ATS, and memory so a week-long recruitment goal does not reset every turn.

What a tool is (and is not)

- A tool is a **function definition** plus a **schema** that teaches Claude the name, when to call it (description), and which JSON arguments to pass.
- The function runs in *your* process: wherever the Anthropic client or Claude Code session is running. The LLM never executes Python on Anthropic's side.

- Typical payloads: current date/time, weather, stock quotes, employee lookups, database reads — anything the weights cannot know.
- A tool is not the same as stuffing the answer into the system prompt. Prompts are static for that turn; tools are invoked when the model judges them relevant.

Trainer vocabulary: “Claude Code” versus Claude

In this session Vijay repeatedly used **Claude Code** to mean the *AI application* (the client that holds functions, builds messages, and talks to the API) and **Claude** to mean the LLM. That is classroom shorthand. In product docs, Claude Code is the terminal/IDE agent; the Messages API client is the Anthropic SDK. For the exam, keep the loop clear: *client executes tools; model only decides*.

Two SDKs, one loop

The cohort had been using `pip install anthropic` (Messages API). The trainer flagged a second stack: the **Claude Agent SDK** (formerly spoken of as Claude Code SDK). Manual tool use makes every handshake visible — stop reason, tool_use block, your if/else, tool_result. The Agent SDK still does that handshake; it just hides the boilerplate so you configure tools and a goal instead of writing the loop.

1. User prompt plus the list of tool schemas goes to Claude.
2. Claude either answers in text or returns a **tool_use** request (stop reason `tool_use`).
3. The application runs the matching function with the model-supplied arguments.
4. The result is sent back as a **user** message containing a **tool_result** block.
5. Claude produces the final natural-language answer (or asks for another tool).

Why not attach every tool to every call

You *can* register many tools; schemas are relatively small. The risk is not token volume so much as **collision**. If two descriptions overlap, Claude may pick the wrong function — a form of tool-level hallucination. If you already know this turn only needs a browser tool, attach that tool. Distinct names and specific descriptions are the control surface the model actually reads.

CLIENT DATA

Do not wire production UST client systems into classroom notebooks. Demo functions (clock, public APIs) are enough to learn the handshake.

How this maps to later chapters

Chapter 02 walks the Messages API handshake in code terms. Chapter 03 treats MCP as the reusable, third-party version of the same idea. Module 11 wraps retrieval as another context technique. Module 12 shows *built-in* Agent SDK tools (Bash, Read, Glob) that you enable rather than write.

Official documentation

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Canonical loop: schemas, tool_use, tool_result.

- **Messages API**

<https://docs.claude.com/en/api/messages>

Where tools are attached to a request.

- **Claude Code documentation**

<https://code.claude.com/docs/en/overview>

Product surface the trainer analogised to “the AI app”.

- **Claude Docs home**

<https://docs.claude.com/en/home>

Index if a recorded notebook path is stale.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.